

Edge Language Coach — Architecture Report

Contents

Edge Language Coach — Architecture Report	3
1. System Overview	3
2. Reliability Mechanisms	3
2.1 Circuit breaker (Groq API)	3
2.2 Rate limiting, retries and idempotency	4
2.3 Timeouts	4
2.4 Recovery Time Objectives	4
2.5 Integration tests	4
3. Scalability	5
3.1 Horizontal gateway scale-out	5
3.2 Measured behaviour	5
3.3 Threats to validity	5
3.4 Worker concurrency	5
3.5 Production deployment	6
4. Observability	6
5. Operator Surface — Agentic MCP	7
5.1 observability-mcp (read only)	7
5.2 remediation-mcp (guarded write)	7
5.3 Capability classification	8
5.4 Why this matters for reliability	8
6. SLO Targets	8
7. Trade-offs	9
8. Future Work	9
C4 Architecture Diagrams	9
Level 1 — System Context	9
Level 2 — Container Diagram	10
Level 2b — Operator Surface (Agentic MCP)	10
Level 3 — Gateway Component Diagram	11
SLO Table	11
Service Level Objectives	11
k6 Load Test Targets	12
Measured Results	12
Reliability Mechanisms vs. Failure Modes	13

Operational Safety Policy	13
1. Capability classification	13
2. Least privilege	14
3. Action boundaries	14
4. Idempotency	15
5. Auditability	15
6. Economic guardrails	15
7. Hallucination and validation	16
8. Rollback and safe fallback	16
9. Approval matrix	16
10. How the policy is enforced	17
Cost, ROI and Return on Agent	17
1. SLOs	18
2. Cost of one hour of downtime	18
3. Justified resilience investment	18
4. Current and projected operational cost	18
Current scale (development, no real users yet)	18
Projected scale (20 active users)	19
When do free tiers stop covering us	19
5. Agent-related cost ceiling	20
6. Cost vs reliability trade-off	20
7. Automation vs safety trade-off	20
8. Return on Agent	21
9. Measurable benefit to customers	21
10. Recovery Time Objectives	21
11. Budget compliance under stress	22
ADR 001 — Node.js Workers over Go Microservices	22
Context	22
Decision	22
Consequences	23
ADR 002 — BullMQ + Redis over RabbitMQ	23
Context	23
Decision	23
Consequences	23
ADR 003 — Supabase over Self-Hosted PostgreSQL	23
Context	23
Decision	23
Consequences	24
ADR 004 — prom-client over Full OpenTelemetry Stack	24
Context	24
Decision	24
Consequences	24

Edge Language Coach — Architecture Report

1. System Overview

Edge Language Coach is an Italian language learning app. It uses LLM-driven conversation practice, spaced-repetition flashcards and automated topic curation to help learners improve fluency.

The system is a distributed, event-driven architecture with three independently deployable runtime units:

Unit	Technology	Role
Web Gateway	React 19 + Vite served by nginx Fastify (Node.js 22)	SPA, static build REST API, auth, reliability primitives
Workers	Node.js + BullMQ	Async jobs (scraper, summary, flashcards)

Full C4 diagrams are in [docs/c4-diagram.md](#).

2. Reliability Mechanisms

2.1 Circuit breaker (Groq API)

The gateway wraps every Groq call (chat completions, Whisper transcription) in an `opossum` circuit breaker. The chat breaker opens after a 50 percent error rate over a rolling 60 s window (minimum 10 requests) and resets after 30 s. The transcribe breaker has the same thresholds but a 60 s timeout (audio inference is slower).

When a breaker is open, requests fail immediately instead of blocking the event loop on a stalled upstream. Breaker state is exposed as a Prometheus gauge (`groq_circuit_breaker_state`) and visible in the Grafana dashboard.

Measured behaviour. We exercised the breaker with a mock Groq server ([mocks/groq-mock.mjs](#)) in slow mode (30 s sleep per request, exceeding the 15 s breaker timeout) and a driver script ([mocks/breaker-driver.mjs](#)) firing 60 requests at concurrency 12. Full output at [load/breaker-demo-output.txt](#).

Phase	Calls	Per-call latency	Status
Closed (probing)	1 to 7	15.3 s (timeout)	breaker collecting failure samples
Transition	call 7		volumeThreshold (10) plus 50 percent error rate crossed, breaker opens
Open (short-circuit)	8 to 60	12 to 58 ms	requests fail fast, never reach mock

Adjacent-call latency cliff: 15,300 ms to 25 ms. Without the breaker all 60 requests would have queued against the slow upstream and paid the 15 s timeout each. With it, only 7 calls paid that cost.

2.2 Rate limiting, retries and idempotency

- **Rate limit.** 60 requests/minute per authenticated user (per IP for unauthenticated traffic), backed by Redis so the limit is shared across gateway replicas. Health probes are allowlisted.
- **Retries.** BullMQ workers use `attempts: 3`, `backoff: { type: 'exponential', delay: 5000 }`. Failed jobs after the retry budget are logged at `error` level with `dlq: true` and visible in Bull Board at `:3002/queues`.
- **Idempotency.** The scraper sets `scraper:slot:<date>:<6h-window>` in Redis on success (TTL 6 h). A retry inside the same window is a no-op. Title-level deduplication against the last 30 days of topics is a second layer.

2.3 Timeouts

Gateway 30 s connection, 5 s keep-alive. Groq chat 15 s, transcribe 60 s. Health probe 2 s per dependency (Redis, Supabase, Groq).

2.4 Recovery Time Objectives

Derived from the configured timeouts and retry policies, not measured separately.

Failure mode	Total RTO
Groq slowdown or outage	about 90 s (breaker opens within one 60 s window, then 30 s <code>resetTimeout</code> , then 2 s <code>/readyz</code> propagation)
Redis blip	about 20 s
Gateway replica crash	sub-second (nginx-lb round-robins to a healthy replica)
Worker job failure	about 20 s before DLQ visibility
Scraper duplicate replay	about 1 s

The Groq RTO dominates because it covers the user-facing chat path.

2.5 Integration tests

The reliability claims above are pinned in CI. The gateway has a Vitest suite ([apps/gateway/src/tests/health.test.ts](#)) using Fastify's `inject()` harness, with Redis, Supabase and Groq mocked at module boundaries so tests run hermetically. Currently covered: `/livez` 200, `/readyz` 200 when healthy, `/readyz` 503 with the failing dep named when Redis or Groq go down.

CI ([.github/workflows/ci.yml](#)) runs `pnpm turbo test` ahead of typecheck and build on every push and PR. Breaker, rate-limit and queue-retry tests are the natural next additions.

3. Scalability

3.1 Horizontal gateway scale-out

The gateway is stateless. Session state lives in Supabase, rate-limit counters live in Redis. Scaling is one flag:

```
docker compose up --scale gateway=3
```

`nginx-lb` sits in front and round-robins via Docker’s built-in DNS. The web SPA and Prometheus both target `nginx-lb:3001`, so adding replicas needs no client-side or scrape-config change.

3.2 Measured behaviour

The k6 script at `load/gateway.js` runs three scenarios back to back: baseline (10 VUs for 30 s), stressed (ramp 0 to 50 VUs over 60 s) and scaled_out (50 VUs for 30 s). We ran it twice, once with one gateway replica and once with three, keeping everything else fixed.

Metric	1 replica	3 replicas	Δ
gateway_errors rate	6.81 %	0.27 %	−96 %
scaled_out p95 latency	2.43 s	2.15 s	−12 %
stressed p95 latency	1.73 s	1.84 s	+6 %
baseline p95 latency	0.34 s	0.66 s	+93 % (cold-start noise, see Section 3.3)
Sustained throughput	39.3 req/s	39.4 req/s	unchanged

Adding replicas had a dramatic effect on error rate (the 1-replica gateway dropped about 7 percent of requests under saturation, the 3-replica setup dropped under 0.3 percent) and a modest effect on latency. The gateway is the bottleneck for availability, the downstream hot path (Supabase RTT, Redis round-trips) is the bottleneck for latency, so adding gateway replicas does not help with the latter.

The persistent ~25 percent `http_req_failed` rate seen in both runs is the rate limiter doing its job. 50 VUs at ~10 req/s is roughly 500 req/min, well over the 60 req/min cap, so most failures are 429s.

3.3 Threats to validity

- **Co-located load generator.** k6 ran on the same Windows host as Docker Desktop, competing for CPU. Most visible in the unexpected **baseline** regression on the 3-replica run.
- **Mocked Groq.** All Groq calls returned in under 1 ms via the mock. Deliberate (isolates the gateway and queue layer) but means absolute p95 numbers are not production estimates.
- **Single iteration per configuration.** No statistical variance.

3.4 Worker concurrency

Workers scale horizontally too. BullMQ supports multiple consumers on the same queue and guarantees each job is processed exactly once. Per-queue concurrency: 3 for **flashcard-generate**

and `summary-generate` (limited by Groq rate limits), 1 for `topic-scrape` (serial to avoid race conditions on dedup state).

3.5 Production deployment

Production splits the system across two hosts. The gateway, workers, Redis and `nginx-lb` run on a single Oracle Cloud VM via [docker-compose.prod.yml](#), pulling multi-arch (`linux/amd64` and `linux/arm64`) images from GHCR. The React SPA runs on Vercel.

Three GitHub Actions workflows handle the pipeline, separated by path filter so a docs-only or web-only change skips the backend rebuild:

Workflow	Trigger	Result
ci.yml	every push and PR	<code>pnpm turbo test</code> plus typecheck plus build. No deploy.
deploy.yml	push to <code>main</code> , paths-ignore <code>web/docs/markdown</code>	tests, then <code>docker buildx</code> multi-arch push to GHCR, then SSH deploy to Oracle
deploy-web.yml	push to <code>main</code> , paths <code>web/packages/vercel.json</code>	tests, then <code>npx vercel --prod</code>

The Oracle deploy step does `git reset --hard origin/main`, `docker compose pull && up -d`, then **explicitly restarts `nginx-lb`** so it re-resolves the new gateway container. Docker's embedded DNS otherwise caches the old IP and the load balancer keeps trying to reach a container that no longer exists.

The Vercel build is driven by [vercel.json](#), which rewrites `/api/:path*` to the Oracle backend, so the browser only ever talks to the Vercel origin. No CORS at the gateway. The prod compose omits Prometheus, Grafana and Bull Board (they remain in `docker-compose.yml` for the local observability story) and the SPA (Vercel handles that).

4. Observability

All services emit JSON-structured logs via `pino` with `service`, `requestId` and relevant domain fields (`sessionId`, `jobId`, `queue`). Log level is configurable via `LOG_LEVEL`.

Both the gateway and the workers expose `/metrics` in Prometheus text format, scraped every 15 s. Key metrics:

Metric	Type	Labels
<code>http_request_duration_seconds</code>	Histogram	method, route, status_code
<code>groq_request_duration_seconds</code>	Histogram	operation, status
<code>groq_circuit_breaker_state</code>	Gauge	breaker
<code>bullmq_job_duration_seconds</code>	Histogram	queue, status
<code>jobs_dead_letter_total</code>	Counter	queue

Metric	Type	Labels
queue_depth_total	Gauge	queue
active_sessions_total	Gauge	

Node defaults (event-loop lag, GC, RSS) are collected via `collectDefaultMetrics`. A Grafana dashboard provisioned from `docker/grafana/dashboards/edge-coach.json` is auto-loaded on startup.

Two health probes back container orchestration. `GET /livez` always returns 200. `GET /readyz` pings Redis, Supabase and Groq with 2 s timeouts and returns 503 with the failing dependency named in the body.

5. Operator Surface — Agentic MCP

Two [Model Context Protocol](#) servers expose the running system to agent-driven inspection and remediation, without SSH, kubectl or a redeploy. The loop is: detect (Prometheus, Grafana) then diagnose (`observability-mcp`) then remediate (`remediation-mcp`).

The two servers are split by privilege.

5.1 observability-mcp (read only)

Wraps `/metrics` and `/readyz` as six typed tools. No mutation, no auth needed (stdio, locally scoped). Source: [apps/observability-mcp/src/index.ts](#).

Tool	Backed by
<code>get_service_health</code>	gateway and workers <code>/readyz</code>
<code>get_queue_metrics</code>	<code>queue_depth_total</code> per queue
<code>get_circuit_breaker_state</code>	<code>groq_circuit_breaker_state{breaker}</code>
<code>get_active_sessions</code>	<code>active_sessions_total</code>
<code>get_groq_latency</code>	<code>groq_request_duration_seconds</code> histogram lines
<code>get_safety_policy</code>	returns <code>docs/safety-policy.md</code> so the agent self-binds at session start

5.2 remediation-mcp (guarded write)

Mutates runtime state through three layered guards (typed input, secret, irreversibility check). Source: [apps/remediation-mcp/src/index.ts](#).

Tool	Mechanism	Guard	Reversible
<code>pause_queue / resume_queue</code>	BullMQ admin via Redis	<code>zod.enum</code> restricts queue name	yes
<code>reset_circuit_breaker</code>	POST <code>/admin/breakers/reset</code> header	<code>ADMIN_API_KEY</code>	yes (may re-open if upstream is still bad)

Tool	Mechanism	Guard	Reversible
<code>flush_dead_letter_queue</code>	<code>Queue.clean('failed')</code>	<code>confirm: true</code> argument required	no

Every call writes a structured pino audit line to stderr (stdout is reserved for the MCP transport).

5.3 Capability classification

Every capability the agent can touch falls into deterministic, agentic or human in the loop. The line is not impact magnitude, it is whether the decision needs interpretation across heterogeneous signals.

Capability	Class	Approval
Breaker trip, BullMQ retry, rate limiter, scraper idempotency	Deterministic	None (threshold based)
<code>get_service_health</code> plus <code>get_groq_latency</code> synthesis	Agentic (advisory)	None (read only)
<code>pause_queue</code> / <code>resume_queue</code> during a drain	Agentic (bounded)	None (reversible, typed enum)
<code>reset_circuit_breaker</code>	Human in the loop	Holder of <code>ADMIN_API_KEY</code>
<code>flush_dead_letter_queue</code>	Human in the loop	Explicit <code>confirm: true</code> from a human
Infrastructure scaling	Operator only	Out of agent scope

Full classification, hallucination controls, economic guardrails and rollback rules are in [docs/safety-policy.md](#). The same numbers are derived in [docs/cost-roi.md](#). At session start the agent calls `observability-mcp.get_safety_policy` and self-binds before any remediation.

5.4 Why this matters for reliability

Without the MCP layer, recovering from an open breaker or a poisoned DLQ needs a Bull Board click-through, a `redis-cli` session or a redeploy. With it, the same primitives are exposed as a typed, audited contract any MCP client can call.

When the breaker demo opens the chat breaker, the recovery is a three-tool dialogue: `get_circuit_breaker_state` to confirm open, verify Groq is healthy, then `reset_circuit_breaker` to close. This partially compensates for the absence of distributed tracing (ADR 004). The operator cannot follow a single span through the system but can interrogate live state with structured tools.

6. SLO Targets

Full SLO table including k6 load test targets and measured results is in [docs/slo-table.md](#). Headlines: API availability at least 99 percent, `/api/messages` p95 under 2 s at baseline (10 VUs), 5xx rate under 1 percent at baseline.

7. Trade-offs

Full reasoning is in the ADRs in [docs/adr](#).

- **Node.js workers over Go microservices.** Shared TypeScript types, same toolchain, faster iteration. Workers cannot scale independently (ADR 001).
 - **BullMQ on Redis over RabbitMQ.** One Redis instance does both queuing and rate limiting. Redis is not a purpose-built broker (ADR 002).
 - **Supabase over self-hosted Postgres.** Saved weeks on auth. External dependency, cannot run fully offline (ADR 003).
 - **prom-client only over full OpenTelemetry.** Faster to instrument, fewer services. No cross-service tracing (ADR 004).
 - **Stateless gateway.** All shared state in Redis. Horizontal scale-out is one flag.
 - **MCP operator surface shipped.** The split between read-only and guarded-write servers is the load-bearing decision. Bundling both into one would have meant every consumer inherits the privilege of the most-privileged tool.
-

8. Future Work

- OpenTelemetry distributed tracing (gateway to workers correlation through the BullMQ payload).
- Kubernetes manifests with HPA on `queue_depth_total` and `http_request_duration_seconds`.
- Redis AOF persistence enabled for production durability.
- Tighten `remediation-mcp` auth so `pause_queue` and `resume_queue` also require `ADMIN_API_KEY`, not just `reset_circuit_breaker`.
- Statistical load testing with multiple iterations per configuration on a dedicated load-generation host.

C4 Architecture Diagrams

Level 1 — System Context

```
graph TB
    user["Language Learner\n(Browser)"]
    system["Edge Language Coach\n[Software System]\nItalian language practice\nvia AI-powered o"]
    supabase["Supabase\n[External System]\nPostgreSQL + Auth"]
    groq["Groq API\n[External System]\nLLM inference (Llama 3.3)\nWhisper transcription"]

    user -->|"Uses (HTTPS)"| system
    system -->|"Reads/writes sessions,\ntopics, flashcards"| supabase
    system -->|"Chat completions,\naudio transcription"| groq
```

Level 2 — Container Diagram

graph TB

```
browser["Browser SPA\n[React 19 + Vite]\nUser interface for chat,\nflashcards, reports"]

nginx_web["Web Server\n[nginx:alpine]\nServes static SPA,\nproxies /api to nginx-lb"]

nginx_lb["Load Balancer\n[nginx:alpine]\nRound-robin across\ngateway replicas"]

gateway["API Gateway\n[Fastify / Node.js]\nAuth, rate limiting,\ncircuit breaker, REST API"]

workers["Workers\n[Node.js / BullMQ]\nScraper, summary,\nflashcard generation\n:3002"]

redis["Redis\n[redis:7-alpine]\nBullMQ job queues,\nrate-limit counters\n:6379"]

prometheus["Prometheus\n[prom/prometheus]\nMetrics scraping\n:9090"]

grafana["Grafana\n[grafana/grafana]\nDashboards & alerting\n:3000"]

supabase[("Supabase\n[External]\nPostgreSQL + Auth")]
groq["Groq API\n[External]"]

browser -->|"HTTPS"| nginx_web
nginx_web -->|"proxy /api"| nginx_lb
nginx_lb -->|"round-robin"| gateway
gateway -->|"enqueue jobs"| redis
gateway -->|"SQL queries"| supabase
gateway -->|"LLM / Whisper\n(circuit-broken)"| groq
workers -->|"consume jobs"| redis
workers -->|"SQL writes"| supabase
workers -->|"LLM inference"| groq
prometheus -->|"scrape /metrics"| nginx_lb
grafana -->|"PromQL queries"| prometheus
```

Level 2b — Operator Surface (Agentic MCP)

A separate plane from the user-request data flow. MCP clients (Claude Desktop, Claude Code, automated runbook agents) speak the stdio transport to two servers split by privilege.

graph LR

```
operator["Operator / Agent\n(Claude Desktop, Claude Code)"]

obs_mcp["observability-mcp\n[stdio MCP server]\nRead-only:\n• get_service_health\n• get_queue_status"]

rem_mcp["remediation-mcp\n[stdio MCP server]\nGuarded write:\n• pause_queue / resume_queue"]

gateway["API Gateway\n:3001"]
workers["Workers\n:3002"]
```

```

redis["Redis"]
admin["POST /admin/breakers/reset\n(ADMIN_API_KEY-guarded)"]
audit["stderr audit log\n(pino, structured)"]

operator -->|"stdio"| obs_mcp
operator -->|"stdio"| rem_mcp

obs_mcp -->|"GET /metrics, /readyz"| gateway
obs_mcp -->|"GET /readyz"| workers

rem_mcp -->|"BullMQ admin"| redis
rem_mcp -->|"x-admin-key"| admin
admin --- gateway
rem_mcp -.->|"every call"| audit

```

Loop closure: detect (Prometheus / Grafana, not shown) → diagnose (observability-mcp) → remediate (remediation-mcp).

Level 3 — Gateway Component Diagram

```

graph TB
    subgraph Gateway ["API Gateway (Fastify)"]
        auth["Auth Plugin\n(JWT validation)"]
        rl["Rate Limit Plugin\n(@fastify/rate-limit + Redis)\n60 req/min per user"]
        cb["Circuit Breaker\n(opossum)\nncat + transcribe breakers"]
        routes["Route Handlers\n/api/topics, /api/sessions,\n/api/messages, /api/flashcards,\n"]
        health["Health Routes\n/livez, /readyz, /metrics"]
        queue_client["Queue Client\n(BullMQ producer)\nflashcard-generate\nsummary-generate"]
    end

    req["Incoming Request"] --> auth --> rl --> routes
    routes --> cb
    routes --> queue_client
    cb -->|"Groq SDK"| groq["Groq API"]
    queue_client -->|"Redis"| redis["Redis"]
    health -->|"ping"| redis

```

SLO Table

Service Level Objectives

SLI	Measurement Method	Target (SLO)	Measurement Window
API avail- ability	1 - rate(http_requests_total{status_code=~"5.."}[5m])	99%	Rolling 5 min

SLI	Measurement Method	Target (SLO)	Measurement Window
/api/messages p95 latency	histogram_quantile(0.95, rate(http_request_duration_seconds_bucket{route="/api/messages"}[5m]))	< 2 s	Rolling 5 min
/api/topics p95 latency	histogram_quantile(0.95, rate(http_request_duration_seconds_bucket{route="/api/topics"}[5m]))	< 500 ms	Rolling 5 min
/readyz avail-ability	Prometheus up metric	100%	Continuous
Queue job success rate	Failed jobs / total jobs processed	95%	Per day
Scraper fresh-ness	Topics inserted per 24 h	5 new topics	Per day

k6 Load Test Targets

Scenario	VUs	Duration	p95 Latency Target	Error Rate Target
Baseline	10	30 s	< 2 s	< 1%
Stressed	ramp 0→50	75 s	< 4 s	< 5%
Scaled-out (2 replicas)	50	30 s	< 2 s	< 1%

Measured Results

Sourced from `k6 run load/gateway.js` against `docker compose up --scale gateway=N` with Groq mocked (see [architecture-report.md](#) for full methodology and threats to validity). Comparison of the same script run against 1 vs 3 gateway replicas:

Scenario	Replicas	p95 latency	gateway_errors rate	Throughput
Baseline	1	0.34 s	n/a	—
Baseline	3	0.66 s	n/a	—
Stressed	1	1.73 s	n/a	—
Stressed	3	1.84 s	n/a	—
Scaled-out	1	2.43 s	6.81 %	39.3 req/s
Scaled-out	3	2.15 s	0.27 %	39.4 req/s

Headline: going from 1 to 3 gateway replicas drops the `scaled_out` error rate by about 96 percent (6.81 to 0.27) with throughput essentially unchanged. The bottleneck moves from the gateway event loop to the downstream Supabase and Redis round-trips. The persistent ~25 percent `http_req_failed` rate is the rate limiter doing its job (50 VUs at ~10 req/s is roughly 500 req/min against a 60 req/min cap).

The unexpected **baseline** p95 regression on the 3-replica run is host-CPU contention from the co-located k6 generator, documented under threats to validity in the architecture report.

Reliability Mechanisms vs. Failure Modes

Failure Mode	Detection	Recovery
Groq API slow/unavailable	Circuit breaker opens after 50% errors in 60 s window	Breaker resets after 30 s; cached response or 503 returned
Redis unavailable	<code>/readyz</code> returns 503; rate-limiter falls back to in-memory	BullMQ reconnects automatically; job queue persists in Redis
Worker job crash	<code>job.failed</code> event logged at <code>error</code> level with <code>dlq: true</code> after 3 attempts	Manual replay via Bull Board UI at <code>:3002/queues</code>
Gateway overloaded	Rate limiter returns 429 after 60 req/min per user	nginx-lb spreads load across replicas; scale with <code>--scale gateway=N</code>
Scraper re-run within same slot	Redis idempotency key checked at job start	Job exits early (<code>skipping - already completed this slot</code>)

Operational Safety Policy

This is the safety policy for the agentic operator surface (the two MCP servers in `apps/observability-mcp` and `apps/remediation-mcp`). It says what the agent is allowed to do, where the boundaries are and what happens when it tries to step outside them.

The policy is written down here, enforced in code at the MCP server boundary and exposed to the agent itself through the `get_safety_policy` tool on `observability-mcp` so the agent self-binds before it ever calls a remediation tool.

1. Capability classification

Every operational capability is either deterministic automation, agentic decision making or human in the loop. The line is not impact magnitude, it is whether the decision needs interpretation across heterogeneous signals.

Capability	Class	Why
Circuit breaker trip, BullMQ retry, rate limiter, scraper idempotency	Deterministic	Threshold over a rolling window or a fixed retry budget. No judgement needed.

Capability	Class	Why
<code>get_service_health / get_groq_latency</code> synthesis	Agentic (advisory)	Answering “is Groq actually healthy now” needs cross-correlating <code>/readyz</code> , the breaker gauge and the latency histogram. No single metric answers it.
<code>pause_queue / resume_queue</code> during a drain	Agentic (bounded)	Reversible. Restricted to a typed enum of known queues. Low blast radius.
<code>reset_circuit_breaker</code>	Human in the loop	Reversible but consequential. Agent proposes, human approves by holding the <code>ADMIN_API_KEY</code> .
<code>flush_dead_letter_queue</code>	Human in the loop	Irreversible. Requires <code>confirm: true</code> from a human in the tool call.
Infrastructure scaling	Operator only	Out of agent scope.

The agentic value sits in the diagnosis step before `reset_circuit_breaker`. Deciding whether to reset is not “if metric X > threshold then act”. It is “the breaker is open, but is Groq actually healthy and is the rest of the system in a state where resetting is safe”. That cross-signal interpretation is what the LLM tool-use loop is good at and what a static runbook would have to enumerate exhaustively to match.

2. Least privilege

The two MCP servers are split by privilege so that an agent connected only to `observability-mcp` has no way to mutate state.

`observability-mcp` is read only over `/metrics` and `/readyz`. No auth needed because no mutation is possible. `remediation-mcp` mutates Redis and the gateway admin route, behind three layers of guard (typed input, secret, irreversibility check).

The reasoning runs in an isolated process. Telemetry is consumed read only. Every write action goes through a typed interface with policy checks in front of it.

3. Action boundaries

Each tool has guards picked so that a riskier action sits behind a stronger barrier.

Tool	Typed input	Secret	Irreversibility check
<code>pause_queue, resume_queue</code>	<code>zod.enum(["flashcard-generate", "summary-generate"])</code> rejects unknown queues	generated	not required (reversible)

Tool	Typed input	Secret	Irreversibility check
<code>reset_circuit_breaker</code>	zod schema on the breaker name	<code>ADMIN_API_KEY</code> HTTP header on the gateway admin route	not required (breaker can re-open if upstream is still bad)
<code>flush_dead_letter_queue</code>	zod enum on queue name	not required	<code>confirm: true</code> argument mandatory; refusal with a typed error without it

Hallucinated tool names, malformed arguments and wrong types are rejected at the boundary by zod before any dispatch. The MCP server cannot execute a tool that does not exist in its registry.

4. Idempotency

Where it matters, operations are no-ops on a duplicate call. `pause_queue` on an already paused queue is a no-op. `reset_circuit_breaker` on an already closed breaker is a no-op. The scraper uses a Redis key with a 6 h TTL so a worker restart mid-scraper does not cause duplicate topic ingestion.

`flush_dead_letter_queue` is not idempotent, which is why it sits behind `confirm: true` rather than relying on idempotency.

5. Auditability

Every call to `remediation-mcp` writes a structured pino line to `stderr` (`stdout` is the MCP transport). The line includes timestamp, tool name, the agent's arguments, the principal and the result.

After an incident, you can `grep stderr` for the sequence of tool calls and see exactly what the agent did and in what order. Read tools on `observability-mcp` do not audit per call to avoid log spam.

6. Economic guardrails

The agent has bounded resource consumption. Past any threshold the agent stops and defers to a human.

Resource	Threshold	What happens
Tool calls per incident	10	Agent returns <code>escalation_required</code> and stops. Operator takes over.
Tool calls per principal per 10 min	20	<code>remediation-mcp</code> returns <code>rate_limit_exceeded</code> . 10 min cooldown.
Inference cost per incident	\$0.50	Computed from token usage in the agent client. Agent stops at the ceiling.
Inference cost per month	\$20	Hard cap in the agent client config. Past this, agentic features are disabled.

Resource	Threshold	What happens
Scaling actions	not permitted	Agent cannot trigger <code>docker compose --scale</code> .

The per-incident tool call ceiling is the load-bearing one. It is the bound that protects against a reasoning loop where the agent keeps calling `get_service_health` without converging. Past 10 calls without a remediation plan, the assumption is that the agent does not have enough signal to act.

Enforcement is layered: the agent client tracks the counters and ceilings, the MCP server applies the per-principal rate limit as a second line. Cost ceilings are tracked client side because the MCP server has no visibility into token usage.

7. Hallucination and validation

The agent can produce a plausible-looking tool call that is wrong. The policy protects against this in three places.

First, zod schemas at the MCP boundary reject hallucinated tool names, misspelled enums and wrong-typed arguments before dispatch. The agent never executes a hallucination because the boundary refuses to dispatch one.

Second, before a high-impact action like `reset_circuit_breaker`, the agent is expected to call `get_service_health` and confirm the relevant dependency reports healthy. The verify-before-act pattern uses live telemetry as the validation surface. If `/readyz` says Groq is down, the breaker reset is refused.

Third, diagnosis happens entirely through `observability-mcp`, which has no mutation tools. The agent cannot accidentally remediate while diagnosing.

We do not run a staging environment for dry runs. The system is small and the cost of a parallel staging copy is not justified.

8. Rollback and safe fallback

The default when the agent is uncertain is to do nothing and escalate. Three cases:

- **Inconsistent telemetry** (e.g. breaker open but `/readyz` says Groq is fine): do not act, escalate with the conflicting signals attached.
- **Per-incident tool call ceiling hit**: stop, return `escalation_required` with the partial diagnosis.
- **Poison-pill job in the DLQ with no clear root cause**: refuse to flush, pause the queue instead and escalate.

The pattern is the same in each case. Bounded reversible action (pause the queue) is preferred over speculative remediation (flush the queue). If the agent cannot reach high confidence on the root cause, it stops and waits for the operator.

9. Approval matrix

Action	Class	Approval
Breaker trip and half-open probe	Fully automated	None (deterministic)
Job retry on transient failure	Fully automated	None (BullMQ policy)
<code>pause_queue</code> / <code>resume_queue</code> during normal operations	Agent autonomous	None (reversible, typed)
<code>reset_circuit_breaker</code>	Agent advisory, operator executes	Holder of <code>ADMIN_API_KEY</code>
<code>flush_dead_letter_queue</code>	Agent advisory, operator executes	Explicit <code>confirm: true</code> from a human
Gateway scale-out	Operator only	Out of agent scope
Schema migration on Supabase	Operator only	Out of agent scope, CI gated

10. How the policy is enforced

Three layers, outermost to innermost.

Agent context. The policy is exposed to the agent through `observability-mcp.get_safety_policy`. The agent calls it at session start and self-binds. This is defence in depth. It fails open if the agent skips the call, which is why the next two layers exist.

MCP server boundary. zod schemas reject malformed or hallucinated tool calls. The privilege split between the two servers means an agent connected only to the read-only server cannot mutate at all. Audit lines on stderr give an after-the-fact trace.

Gateway admin route. `POST /admin/breakers/reset` checks `ADMIN_API_KEY` and refuses without it. Last line, behind the MCP server, the one that holds when the other two fail.

If the professor acts as a confused or malicious operator during the demo (which the SRS warns is possible), each layer refuses independently. The intended sequence:

- Try `flush_dead_letter_queue` without `confirm: true`. Layer 2 refuses.
- Try `pause_queue` on a queue that does not exist. Layer 2 refuses via the zod enum.
- Try `reset_circuit_breaker` without `ADMIN_API_KEY`. Layer 3 refuses with 401.
- Ask the agent to flush the DLQ in an ambiguous situation. Layer 1 refuses and proposes `pause_queue` instead.

Each refusal leaves a structured trace.

Sources: [apps/observability-mcp/src/index.ts](#), [apps/remediation-mcp/src/index.ts](#), [apps/gateway/src/routes/admin](#)
The cost ceilings in Section 6 are derived in [docs/cost-roi.md](#).

Cost, ROI and Return on Agent

The point of this document is to show that the engineering choices were made with cost in mind, not just technical merit. The system currently runs entirely on free tiers. Section 4 walks through what changes when we scale to 20 active users and what changes again at the point where free tiers stop covering us.

1. SLOs

In [docs/slo-table.md](#). Headlines: API availability at least 99 percent, `/api/messages` p95 under 2 s at baseline, queue job success at least 95 percent per day, at least 5 fresh topics per day.

2. Cost of one hour of downtime

This is a language coaching app, not fraud detection. We are honest that the absolute downtime cost is small.

At our current scale (no paying users yet), direct revenue loss is zero. The cost is reputational and habit-breaking. Language learning depends on daily streaks; an outage during someone's evening session is the kind of thing that ends the habit.

If we project a 100 paying user scale, the downtime cost is roughly \$20 to \$60 per hour. About \$1/h of direct revenue loss plus \$10 to \$50 of churn risk (5 percent baseline monthly churn, a 10 percent lift on the cohort that saw the outage, \$10/mo LTV scaled over 12 months). Still small, which is why the resilience investment in Section 3 stays proportional.

The same primitives applied to one of the SRS example domains (fraud-alert triage, hospital escalation) would have hourly downtime costs in the thousands. The substrate transfers, the cost regime does not.

3. Justified resilience investment

The single largest resilience investment is the 3-replica gateway behind `nginx-lb`. From the architecture report:

Configuration	Cost on Oracle free tier	Error rate under saturation
1 gateway replica	\$0	6.81 percent
3 gateway replicas	\$0	0.27 percent

A 96 percent error rate reduction for zero extra cost. On a paid provider this would cost roughly \$10 to \$20/mo extra (two small containers). At the low end of our projected downtime cost (\$20/h), the 3-replica investment pays for itself after about one avoided incident per month.

4. Current and projected operational cost

Current scale (development, no real users yet)

Everything is on free tiers. Real numbers, not estimates:

Service	Tier	Cost	Headroom
Oracle Cloud VM (gateway, workers, Redis, <code>nginx-lb</code>)	Always Free	\$0	4 ARM cores, 24 GB RAM
Vercel (SPA hosting)	Hobby	\$0	100 GB bandwidth/mo

Service	Tier	Cost	Headroom
Supabase (Postgres + auth)	Free	\$0	500 MB DB, 2 GB egress/mo
GitHub Actions (CI/CD)	Free for public repos	\$0	2,000 minutes/mo
Groq API (LLM)	Pay as you go	\$0 to \$1	a few hundred dev requests
Total		\$0 to \$1/mo	

We pay essentially nothing right now.

Projected scale (20 active users)

20 users at 5 sessions a week, 15 messages per session, 500 tokens per message. That is about 400 sessions and 3M tokens a month.

Service	Tier	Cost	Notes
Oracle Cloud VM	Always Free	\$0	Still inside 4 cores / 24 GB. Plenty of headroom.
Vercel	Hobby	\$0	20 users will not get close to the 100 GB bandwidth cap.
Supabase	Free	\$0	DB size grows with sessions and flashcards but stays well under 500 MB.
GitHub Actions	Free	\$0	Same CI volume regardless of user count.
Groq API	Pay as you go	about \$2 to \$3	3M tokens at \$0.59/\$0.79 per million for input/output, split roughly 50/50.
Total		about \$2 to \$4/mo	

The cost grows almost entirely with Groq usage. Everything else stays free at this scale.

When do free tiers stop covering us

The thresholds, roughly:

- **Supabase free tier** runs out around 200 to 500 users, depending on how much session history we retain. Egress is the usual first limit. Pro tier is \$25/mo.

- **Oracle Always Free VM** is comfortable up to a few hundred users at this load profile. Past that we would either move to a paid Oracle shape (\$20 to \$40/mo) or split workers onto a second VM.
- **Vercel hobby** caps at 100 GB egress/mo. The SPA is small, so this is unlikely to bite before Supabase does.
- **Groq** has no tier, you just pay per token. At 1,000 users the LLM cost is about \$100 to \$150/mo on its own.

A reasonable estimate for the system running at 1,000 users: \$150 to \$250/mo all in. Still under any normal SaaS infrastructure budget.

5. Agent-related cost ceiling

The agent burns tokens on the client side (Claude Desktop, Claude Code or another MCP client) when it calls our tools. The MCP servers themselves do not call any LLM.

Bound	Threshold
Tool calls per incident	10
Tool calls per principal per 10 min	20
Inference cost per incident	\$0.50
Inference cost per month	\$20

Past the per-incident ceiling, the agent stops and escalates. Past the monthly ceiling, agentic features are disabled and the operator falls back to manual diagnosis (Bull Board, `redis-cli`, raw Grafana). Per-month math: 4 incidents at \$0.50 each is \$2/mo in normal operation. The \$20 ceiling is sized for an unusual month with extended diagnosis sessions.

Full enforcement detail is in [docs/safety-policy.md](#).

6. Cost vs reliability trade-off

We picked **3 retry attempts on BullMQ workers, not 10**.

More retries absorb more transient failures, but a poison-pill job (malformed payload, deleted user, deterministic failure) costs the same compute every retry without ever succeeding. At 10 attempts with 5 s exponential backoff, a poison-pill consumes about 85 minutes of retry budget before reaching the DLQ. At 3 attempts, that drops to about 15 seconds.

We chose to surface persistent failures sooner. The cost saved is compute on poison pills. The reliability cost is the rare third-time-successful transient failure that we now miss.

7. Automation vs safety trade-off

We picked **pause_queue agent autonomous, flush_dead_letter_queue human approved**.

Making both agent autonomous would shave minutes off MTTR. But DLQ flush is irreversible. If the root cause was a worker bug that has now been fixed, those jobs could have been replayed instead of discarded.

The line is reversibility, not impact magnitude. Reversible actions can be agent autonomous because the operator can undo. Irreversible actions need a human in the path because the cost of getting it wrong is unbounded.

8. Return on Agent

Without the MCP layer, recovering an open Groq breaker means: get paged, SSH to the Oracle VM, tail the gateway logs, check breaker state from the metrics endpoint by hand, decide whether Groq is healthy, `curl` the admin route with the key, watch the next minute of metrics. About 8 to 12 minutes per incident.

With the MCP layer: get paged, ask the agent or open the Inspector, agent calls `get_circuit_breaker_state` and `get_service_health` and `get_groq_latency`, reports back, proposes the reset, operator approves. About 2 to 4 minutes per incident.

Time saved is roughly 6 minutes per incident. At 4 incidents a month and a \$30/h ops rate, that is about \$12/mo saved. Agent inference cost runs about \$2 to \$5/mo. Net Return on Agent is about \$7 to \$10/mo, a 3 to 5 times return on inference cost.

The dollar number is small because the absolute scale is small. The ratio is what matters; it does not change at higher scale.

Two harder-to-quantify benefits do not show up in the dollar math. First, the audit trail is automatic: every remediation call writes a structured pino line on stderr. The same audit through SSH would require operator logging hygiene. Second, the same MCP tools serve a human (Inspector), an agent (Claude) and a future runbook script. Onboarding a new operator means showing them the Inspector once, not teaching them `redis-cli` and the admin route and the Bull Board UI separately.

9. Measurable benefit to customers

Benefit	How it shows up
Conversational practice with an LLM tutor	<code>active_sessions_total</code> metric
Spaced-repetition flashcards generated per session	Flashcard rows in Supabase, queue throughput in Bull Board
Fresh topics every 6 hours	Scraper freshness SLO (5 new topics per day)
Service availability during a daily session	API availability SLO (99 percent)
Recovery from a Groq outage in under 90 s	RTO target (see Section 10)

10. Recovery Time Objectives

Derived from the configured timeouts and retry policies, not measured separately.

Failure mode	Total RTO
Groq slowdown or outage	about 90 s
Redis blip	about 20 s
Gateway replica crash	sub-second (nginx-lb round-robins to a healthy replica)

Failure mode	Total RTO
Worker job failure	about 20 s before DLQ visibility
Scraper duplicate replay	about 1 s

The 90 s Groq RTO dominates because it covers the user-facing chat path. The other modes are either invisible to the user or fall inside one polling interval.

11. Budget compliance under stress

The system has no autoscaling, which means cost cannot run away from a stress event by construction.

Stress	What happens	Cost impact
Traffic spike to 6x baseline	Rate limiter returns 429s past 60 req/min per user.	None.
Queue backlog from a slow worker	BullMQ holds the backlog in Redis. Workers process serially.	None. Latency degrades gracefully.
Groq tail latency	Circuit breaker opens, requests fail fast.	Spend goes down (failed calls are not billed).
Agent reasoning loop	Per-incident tool call ceiling fires at 10 calls.	Capped at \$0.50 per incident, \$20 per month.
Scaling action requested by agent	Refused. Scaling is out of agent scope.	None.

The trade-off is real: we cannot absorb a 10x traffic spike without manual intervention. We accept that for a course-scale project.

ADR 001 — Node.js Workers over Go Microservices

Date: 2026-04-30 **Status:** Accepted

Context

The original plan was three independently deployable Go services (Topic Scraper, Recommendations Engine, Flashcard/SRS Engine) talking to the gateway over internal REST. The goal was to demonstrate a distributed system with independently scalable components.

Decision

Implement the workers as a single Node.js service (**apps/workers**) using BullMQ on Redis instead of three Go services. The three logical workers (scraper, summary, flashcard) stay isolated as separate **Worker** instances with their own queues and concurrency settings.

Consequences

Positive: one language runtime, shared types via `@edge/shared`, BullMQ provides at-least-once delivery and retry and DLQ for free, and there is no per-service Dockerfile or CI step to maintain.

Negative: workers cannot scale independently from each other in Docker (one service scales all three, mitigated by per-queue concurrency). A crash in one worker process affects all three (mitigated by job persistence in Redis and BullMQ retries). Less dramatic polyglot story for the demo.

ADR 002 — BullMQ + Redis over RabbitMQ

Date: 2026-04-30 **Status:** Accepted

Context

The async event-driven layer needed a message broker between the gateway and the workers (summary generation, flashcard generation, topic scraping). Main candidates were RabbitMQ (AMQP) and BullMQ on Redis.

Decision

Use BullMQ with Redis as the broker.

Consequences

Positive: one dependency. Redis is already the rate-limiter store (`@fastify/rate-limit`) so we reuse it as the queue backend. BullMQ provides retry with exponential backoff, dead-letter tracking, delayed jobs, recurring crons and Bull Board out of the box, all of which would be plugins or custom code in RabbitMQ. Local dev is just `docker compose up redis`. BullMQ is TypeScript-native, so job data is type-safe without protobuf or AMQP schema files.

Negative: Redis is not a purpose-built broker. Under extreme load it can lose acknowledged-but-not-persisted messages without AOF persistence (mitigated by enabling AOF in production). No native fanout or pub-sub (not required here). Switching to RabbitMQ later means replacing all queue producers and consumers.

ADR 003 — Supabase over Self-Hosted PostgreSQL

Date: 2026-04-30 **Status:** Accepted

Context

The system needs a relational database (user sessions, topics, flashcards, feedback) and an authentication provider. Options: self-hosted Postgres plus a separate auth service (Auth.js, Keycloak) or a managed BaaS like Supabase.

Decision

Use Supabase (managed Postgres plus Auth plus Storage).

Consequences

Positive: auth is out of the box. Row-level security, JWT issuance, OAuth and email/password are available without an extra service. The gateway validates Supabase JWTs directly, no auth microservice. No Postgres container to manage, tune, back up or upgrade. The free tier comfortably fits a course demo. Prisma works against Supabase identically to self-hosted Postgres, so a switch later is a connection string change.

Negative: external dependency, so the system cannot run fully offline. Vendor lock-in on the auth side (Supabase-specific JWT claims and row-level security policies would need migration). Network latency to Supabase varies; mitigated by Prisma connection pooling and regional deployments.

ADR 004 — **prom-client** over Full OpenTelemetry Stack

Date: 2026-04-30 **Status:** Accepted

Context

The observability step required metrics and dashboards. Two approaches: a full OpenTelemetry stack (traces plus metrics plus logs through an OTEL Collector) or **prom-client** (Prometheus metrics only) with Grafana.

Decision

Use **prom-client** to expose Prometheus-format metrics at `/metrics`. Self-host Prometheus and Grafana in Docker Compose. Defer distributed tracing as an optional future enhancement.

Consequences

Positive: low dependency count. **prom-client** is a single npm package, no OTEL Collector sidecar, no OTLP endpoint, no Jaeger or Tempo or Zipkin. Metrics flow directly into Prometheus and Grafana queries it natively, so the whole pipeline is four Compose services (gateway, prometheus, grafana, nginx-lb). Histogram buckets give p50/p95/p99, counters track error rate and volume, which is all we need for the SLO table. Instrumentation is a Fastify `onResponse` hook, about 10 lines.

Negative: no distributed traces. Spans cannot be correlated across gateway-to-worker hops, so a slow summary job cannot be pinned to a specific Groq call without adding OTEL later. Pull-based scraping at 15 s is coarser than push-based OTEL. Workers do not expose their own `/metrics` endpoint, so queue depth goes through the gateway and adds a query per scrape.